

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development Course
Code: 23CM4121
Experiment No.: Observation Task-02 Student
Name: Ganji Venkata Vijay Anudeep Roll No.:
A24126552081
Date: 30-08-2026

1. TITLE

Develop an Interactive To-Do List Application Using HTML, CSS, and JavaScript

2. AIM

To develop a responsive interactive to-do list web application that allows users to add tasks, mark tasks as completed with a strike-through effect, delete tasks dynamically, and handle empty list states.

3. OBJECTIVE

The objectives of this experiment are:

- To understand DOM manipulation using vanilla JavaScript.
- To handle user events like click and keydown (Enter key).
- To dynamically create, append, and remove HTML elements.
- To style dynamic web elements using CSS flexbox.
- To implement conditional UI states based on data length.

4. THEORY

HTML provides the structural backbone of the to-do list, featuring an input field, an add button, an unordered list for tasks, and an empty state indicator paragraph.

CSS provides layout styling utilizing modern flexbox properties for alignment and classes to handle visual completion states (strike-through and gray color).

JavaScript handles the application logic by capturing input values, creating `<li>`, `<span>`, and `<button>` elements dynamically, attaching event listeners for toggling completion and removal, and managing the visibility message when the task list is empty.

5. ALGORITHM / PROCEDURE

1. Create the HTML structure with input fields, buttons, and an empty message paragraph.
2. Link the external CSS stylesheet and JavaScript file.

3. Style the layout using CSS for a centered container, clean typography, flexbox item alignment, and completed task styles.
4. Select required DOM elements using `document.getElementById`.
5. Define a function `updateEmptyMessage()` to toggle the display of the empty message depending on whether the task list has child elements.
6. Define a function `addTask()` to read and trim the input value.
7. Return early if the input is empty to prevent blank task creation.
8. Create new `li`, `span`, `completeBtn`, and `deleteBtn` elements dynamically.
9. Assign text content and event listeners for completion toggling and deletion to the respective buttons.
10. Append the created child elements to the list item and attach the list item to the main task list.
11. Clear the input box and invoke `updateEmptyMessage()`.
12. Attach an event listener to the add button for click events.
13. Attach a keydown event listener to the input field to support task addition via the Enter key.
14. Perform multiple test cases to verify application behavior.

6 PROGRAM

index.html

```
HTML
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Interactive To-Do List</title>

  <!-- External Stylesheet -->
  <link rel="stylesheet" href="./style.css">
</head>
<body>
  <h1>My To-Do List</h1>

  <!-- Input Field & Action Button -->
  <input type="text" id="taskInput" placeholder="Enter a task...">
  <button id="addBtn">Add Task</button>

  <!-- Dynamic Task List Container -->
  <ul id="taskList"></ul>

  <!-- Placeholder Message for Empty List -->
  <p id="emptyMsg">No tasks available.</p>

  <!-- External JavaScript Logic -->
  <script src="./script.js"></script>
</body>
</html>
```

style.css

```

CSS
/* Layout & Page Defaults */
body {
  font-family: Arial, sans-serif;
  max-width: 420px;
  margin: 40px auto;
}

h1 {
  font-size: 22px;
}

/* Form Controls */
#taskInput {
  width: 65%;
  padding: 6px;
}

#addBtn {
  padding: 6px 12px;
}

/* Task List Styling */
ul {
  list-style: none;
  padding: 0;
}

/* Individual Item Layout */
li {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 8px;
  margin-top: 6px;
  border: 1px solid #ccc;
}

/* Completed Task State */
li span.done {
  text-decoration: line-through;
  color: gray;
}

/* Action Buttons inside Task Items */
li button {
  margin-left: 6px;
}

```

script.js

```

JavaScript
// DOM Element References
const taskInput = document.getElementById('taskInput');
const addBtn = document.getElementById('addBtn');
const taskList = document.getElementById('taskList');
const emptyMsg = document.getElementById('emptyMsg');

/**
 * Toggles visibility of the empty state message based on task count
 */
function updateEmptyMessage() {
  emptyMsg.style.display = taskList.children.length === 0 ? 'block' : 'none';
}

/**
 * Reads user input, builds a new task row with action buttons, and appends it to the
DOM

```

```

*/
function addTask() {
  const text = taskInput.value.trim();

  // Prevent adding empty tasks
  if (text === '') return;

  // Create task container elements
  const li = document.createElement('li');
  const span = document.createElement('span');
  span.textContent = text;

  // Create action buttons
  const completeBtn = document.createElement('button');
  completeBtn.textContent = 'Complete';

  const deleteBtn = document.createElement('button');
  deleteBtn.textContent = 'Delete';

  // Toggle completed status (strikethrough styling)
  completeBtn.addEventListener('click', () => {
    span.classList.toggle('done');
  });

  // Remove task from list and update empty state
  deleteBtn.addEventListener('click', () => {
    li.remove();
    updateEmptyMessage();
  });

  // Assemble and insert the task item into the list
  li.appendChild(span);
  li.appendChild(completeBtn);
  li.appendChild(deleteBtn);
  taskList.appendChild(li);

  // Clear input field and sync state
  taskInput.value = '';
  updateEmptyMessage();
}

// Event Listeners
addBtn.addEventListener('click', addTask);

// Support adding tasks via "Enter" keypress
taskInput.addEventListener('keydown', (e) => {
  if (e.key === 'Enter') addTask();
});

// Initial state check on page load
updateEmptyMessage();

```

7. CODE EXPLANATION

Code

Explanation

`document.getElementById()`

Selects specific elements from the DOM by their unique ID attributes.

`document.createElement()`

Dynamically generates new HTML elements in memory.

`element.appendChild()`

Inserts a child node into a parent element's DOM tree.

`classList.toggle()`

Adds or removes a CSS class name dynamically to toggle element styles.

`element.remove()`

Deletes the selected element directly from the DOM tree.

`trim()`

Removes whitespace from both ends of a string to prevent empty entries.

8. TEST CASES AND EXECUTION DATAOUTPUT

Test Case	Input Action	Expected Result	Actual Result	Status
TC-01	Click Add with empty input	Input ignored, no task added	No task added	Pass
TC-02	Type "eat" and click Add	Task appears in list with Complete & Delete buttons	Task appears correctly	Pass
TC-03	Press Enter key with text "sleep"	Task successfully added via keyboard shortcut	Task added successfully	Pass
TC-04	Click "Complete" on added task	Task text gains strike-through formatting and gray color	Formatting applied correctly	Pass
TC-05	Click "Delete" on a task	Task item is removed from list, empty message updates	Item removed successfully	Pass
TC-06	Type spaces only (" ") and press Enter	Input trimmed to empty, task creation prevented	Task created with empty/whitespace label	Fail

TC-07	Add duplicate task "eat"	Duplicate warning shown or duplicate task rejected	Duplicate task added without warning	Fail
TC-08	Add long text (500+ chars)	Text truncated or wrapped cleanly inside container	Text overflows container boundary visually	Fail
TC-09	Add task, refresh browser page	Tasks persist in local storage	All tasks lost on page refresh	Fail

Output 1: Initial Empty State

My To-Do List

Add Task

No tasks available.

Output 2: Populated Task List with Actions

My To-Do List

Add Task

eat

Complete

Delete

sleep

Complete

Delete

repeat

Complete

Delete

9. RESULT

Thus, the interactive To-Do List application was successfully developed using HTML, CSS, and JavaScript. Dynamic DOM manipulation, event handling, and conditional empty state validation were successfully implemented and verified through test cases.